

Laborator 8 PA

translate, maketrans

str.translate(tabel_traducere)

- tabel_traducere – de obicei obținut cu str.maketrans(x[, y[, z]])

(sau dicționar cu coduri Ascii)

str.maketrans(x[, y[, z]])

- x - dicționar sau șirul cu caracterele de înlocuit
- y – șirul cu caracterele noi (de aceeași lungime cu x) => caracterul x[i] se va înlocui cu y[i]
- z – șir cu caracterele care vor fi șterse

EXEMPLE

1. Se citește un text conținând separatorii uzuali(,;:) Sa se înlocuiască toți separatorii cu spațiu.
2. Se citește un cuvânt format cu litere mici. Să se înlocuiască fiecare vocală din cuvânt cu următoarea literă din alfabet.
3. Aceeași cerință ca la 2, dar în plus să se ștergă semnele: virgula, punct, două puncte.
4. Se citește o propoziție. Să se înlocuiască fiecare cifră < 5 care apare în text cu denumirea ei (1-unu, 2-doi, 3- trei, 4 -patru)

Regex (Regular Expressions)

- *modul de lucru cu expresii regulate (Regular Expressions)*
- Ce sunt **expresiile regulate**?
 - **microlimbaj specializat** încorporat în Python și pus la dispoziție în modulul **re**
 - utilizat pentru a **detecta tipare în stringuri**, pentru a **înlocui acele tipare** cu alte stringuri, pentru a **selecta un string după anumite tipare**, etc
 - limbaj relativ mic și limitat: nu orice sarcina de procesare de stringuri poate fi realizata folosind expresii regulate
- **Anatomia** unei expresii regulate (**RE**)
 - alcătuită din **caractere simple** și **metacaractere**
 - **caracterele simple** se reprezinta pe ele inele, fara vreo alta semnificatie suplimentara
 - **metacaracterele** ajuta la definirea unor tipare mai complexe, detaliate mai jos
 - lista completa a metacaracterelor: `. ^ $ * + ? { } [ ] \ | ( )` (acestea trebuie escape-uite de un backslash)

Documentația pentru modulul de expresii regulate se regăsește la acest link:
<https://docs.python.org/3/library/re.html>

- Un software utilitar foarte bun pentru a testa diverse Regex-uri se regăsește aici: <https://regex101.com/>
- Un cheatsheet cu toate regulile de definire ale unei expresii regulate se regăsește aici: https://images.datacamp.com/image/upload/v1665049611/Marketing/Blog/Regular_Expressions_Cheat_Sheet.pdf

De ce sunt bune aceste expresii regulate? Să zicem ca avem de citit dintr-un fișier sau de la standard input un text care conține date calendaristice, cum am putea extrage aceste informații?

- Putem formula o expresie regulată care descrie această structură, tipar de informație (de obicei o dată calendaristică este structurată sub forma zz.ll.aaaa). Expresia regulată poate fi următoarea: `\d{2}\.\d{2}\.\d{4}`. Bineînțeles că această expresie regulată **NU** garantează corectitudinea informației extrase (i.e. Regex-ul anterior validează următorul șir de caractere 99.99.9999, care nu este o dată calendaristică validă). Astfel am putea folosi puterea acestor expresii regulate doar pentru extragerea informației, urmând să validăm aceste valori separat.

Exemple:

- **split** (împărțirea unei propoziții după o mulțime separator, după o combinație de separatori dintr-o mulțime, după orice caracter care nu este cifră este separator (+combinații) pentru a obține cuvinte care sunt numere, după orice caracter care nu este literă sau semn de punctuație este separator (+combinații))
- **sub** (înlocuirea unui cuvânt cu alt cuvânt, înlocuirea unui cuvânt cu alt cuvânt doar dacă este precedat de un anumit șir)
- **findall - căutare** (cuvintele care încep cu litera A sau a, șirurile binare din text, șirurile binare din text de lungime minim 3, numerele dintr-un text, numerele care se continuă cu “ lei” pentru a le face suma)
- **match vs findall**
 - detectarea datei dintr-un text de forma de forma:
Nume eveniment - data: detalii eveniment

Probleme

1. Verificați în documentația Python 3.x, modulul `re`, funcțiile **compile**, **search**, **sub**, **findall**, **match**, **group**, **start**, **end** și evidențiați ce rol au.
2. Fiind dat un fișier **log.txt** care conține pe fiecare linie anumite acțiuni înregistrate într-un sistem informatic făcute de utilizatorii care au acces la el sau proceduri rulate automat fără intervenția utilizatorilor trebuie să:
 - a. Determinați numărul de acțiuni făcute de utilizatori (i.e. **fără** proceduri rulate automat). (în cazul fișierului prezentat ca exemplu programul ar trebui să afișeze 3).

- b. Pentru fiecare acțiune se regăsește data și ora la care aceste acțiuni au fost făcute. Într-un fișier **output_logs.txt** scrieți toate coordonatele temporare la care un utilizator a făcut o anumită acțiune în sistemul informatic.

log.txt	output_logs.txt
John Doe (john.doe@test.com) [16.11.2024, 10:20 AM]: ran a program on the system John Doe (john.doe@test.com) [16.11.2024, 12:20 PM]: started a new process Automation Task [16.11.2024, 01:20 PM]: backing up database Tim Doe (tim.doe@test.com) [17.11.2024, 09:32 AM]: logged into the system	John Doe: 16.11.2024, 10:20 AM; 16.11.2024, 12:20 PM Tim Doe: 17.11.2024, 09:32 AM

3. <https://www.hackerrank.com/challenges/ip-address-validation/problem>
4. <https://www.hackerrank.com/challenges/html-attributes/problem>
5. <https://www.hackerrank.com/challenges/detect-the-domain-name/problem>

Datetime

Modulul **datetime** din Python oferă clase și funcții pentru a lucra cu date și ore. Este unul dintre cele mai utilizate module standard pentru manipularea datelor calendaristice și temporale. Clasele principale sunt:

- **datetime.date**: pentru lucrul cu date calendaristice (an, lună, zi).
- **datetime.time**: pentru lucrul cu ore (oră, minut, secundă, microsecundă).
- **datetime.datetime**: combină data și ora într-un singur obiect.
- **datetime.timedelta**: pentru diferența între două date sau ore.
- **datetime.tzinfo** și **datetime.timezone**: pentru lucrul cu fusuri orare.

Pentru a lucra doar cu ore se poate utiliza doar modulul **time**.

Exerciții

1. Scrieți un program care să citească datele dintr-un fișier **input_event.txt** cu evenimente, să le proceseze folosind modulul **datetime** și să genereze un raport într-un fișier de output denumit **raport_event.txt**. Raportul trebuie să conțină:
 - Numele evenimentului
 - Data și ora evenimentului formate ca "DD/MM/YYYY HH:MM"
 - Diferența în zile între data curentă și data evenimentului
 - Un mesaj dacă evenimentul a trecut deja: **"Eveniment trecut"**

input_event.txt	raport_event.txt
Aniversare, 20-12-2024 18:00 Concert, 05-12-2024 20:00 Sesiune de examene, 01-01-2025 09:00 Conferință, 25-11-2024 10:30	Aniversare - 20/12/2024 18:00 - Zile până la eveniment: 33 Concert - 05/12/2024 20:00 - Zile până la eveniment: 18 Sesiune de examene - 01/01/2025 09:00 - Zile până la eveniment: 47 Conferință - 25/11/2024 10:30 - Eveniment trecut

2. <https://www.hackerrank.com/challenges/python-time-delta/problem>
3. Să presupunem că un restaurant dorește să gestioneze rezervările pentru mesele sale. Rezervările sunt stocate într-un fișier text "**rezervari.txt**". Fiecare linie din fișier conține detalii despre o rezervare în formatul: **Nume_client, Data_rezervare, Ora_inceput, Durata (în minute)** Trebuie să creăm un program care să:
 - a. Citească datele din fișierul "rezervari.txt".
 - b. Verifice dacă o rezervare a trecut sau nu, în funcție de data și ora curentă.
 - c. Calculeze ora de încheiere a fiecărei rezervări.
 - d. Găsească toate rezervările care se suprapun (au loc în același interval de timp) și să le noteze în fișierul "**raport_rezervari.txt**".
 - e. Genereze un raport cu următoarele informații:
 - Nume client
 - Data și ora de început a rezervării
 - Ora de sfârșit a rezervării
 - Mesaj: "Eveniment în desfășurare", "Eveniment viitor" sau "Eveniment trecut"
 - Suprapuneri cu alte rezervări (dacă există)

rezervari.txt	raport_rezervari.txt
Maria Popescu, 20-11-2024, 18:00, 120 Ion Ionescu, 19-11-2024, 20:30, 90 Ana Georgescu, 20-11-2024, 12:00, 60 Alex Marin, 18-11-2024, 21:00, 150	Maria Popescu - 20/11/2024 18:00 - 20:00 - Eveniment viitor - Suprapuneri cu: Ion Ionescu Ion Ionescu - 19/11/2024 20:30 - 22:00 - Eveniment viitor Ana Georgescu - 20/11/2024 12:00 - 13:00 - Eveniment viitor Alex Marin - 18/11/2024 21:00 - 23:30 - Eveniment trecut